

Product Backlog — Agentic Educational Platform

Product Overview

A web application where students and professors enter technical topics through a **search bar**, and the platform generates a curated learning path, practice problems, and foundational coding tasks powered by an **Agentic AI Orchestrator**.

Epic 1: Authentication, Roles, and Search-Based Topic Discovery

Story 1.1 — User Registration and Login

User Value Statement

As a student or professor, I want to securely create an account and log in, so that I can access personalized learning paths and saved activity.

Acceptance Criteria

- Users can register with email and password.
- Users can log in and log out securely.
- Users can reset their password.
- Protected pages are accessible only to authenticated users.
- User sessions persist securely across refreshes.

Technical Notes

- AI agent workflows must run only in authenticated user context.

- User identity should be attached to orchestration jobs for traceability and personalization.
 - Role metadata should be available to the orchestrator for future logic branching.
-

Story 1.2 — Role and Profile Setup

User Value Statement

As a user, I want to define whether I am a student or professor and set my experience level, so that the platform can personalize recommendations.

Acceptance Criteria

- Users can choose a role during onboarding.
- Users can select an academic or skill level.
- Users can later edit these settings from profile/preferences.
- Profile settings affect recommendation difficulty and ranking.

Technical Notes

- Profile data should be passed into ranking and task-generation agents.
 - Difficulty-ranking logic should be conditioned on user academic level.
 - Persistent profile data should be stored separately from transient session state.
-

Story 1.3 — Search Bar for Topic Queries

User Value Statement

As a learner, I want to type a technical topic into a search bar, so that I can quickly generate a curated learning path.

Acceptance Criteria

- The dashboard contains a prominent search bar for topic queries.
- Users can type topics such as “Dynamic Programming”, “Graphs”, or “Backtracking”.
- Pressing Enter or clicking Search starts the orchestration workflow.
- Empty submissions are blocked with clear validation messaging.
- The entered topic remains visible after submission.
- The search bar is usable on desktop and mobile layouts.

Technical Notes

- The raw search bar query should be the primary input for the AI orchestrator.
 - Query preprocessing should normalize the topic into canonical concepts and aliases.
 - Spelling-tolerant matching and synonym expansion should be supported.
 - The submitted query should be stored in session state for reuse by downstream modules.
-

Story 1.4 — Search Progress and Workflow Status

User Value Statement

As a learner, I want to see the progress of my topic search after using the search bar, so that I understand what the system is doing.

Acceptance Criteria

- After a search is submitted, the user sees a loading state.
- Progress states are shown for search, validation, ranking, and result assembly.
- Errors are communicated clearly if the workflow fails.
- The interface prevents duplicate submissions while the workflow is running.

Technical Notes

- The orchestrator should expose stage-by-stage execution status.
 - Workflow state should be available to the frontend via structured status payloads.
 - Retries and partial failures should be represented distinctly from success states.
-

Epic 2: Intelligent Learning Material Retrieval

Story 2.1 — Search Internal Database from the Search Bar Query

User Value Statement

As a learner, I want the system to first search internal curated materials based on what I typed in the search bar, so that I receive trusted resources quickly.

Acceptance Criteria

- After a topic is submitted through the search bar, the system first searches the internal DB.
- Internal results can include videos, articles, books, and websites.
- Results are filtered for relevance to the typed topic.
- If internal coverage is sufficient, the platform can return results without external fallback.

Technical Notes

- Retrieval should use metadata, tags, prerequisites, and vector embeddings.
- Matching must support fuzzy and semantic topic similarity, not only exact keywords.
- The system should define configurable sufficiency thresholds based on count, confidence, and source diversity.

Story 2.2 — Fallback to Web Discovery if Internal Results Are Insufficient

User Value Statement

As a learner, I want the platform to search the web when internal results are too few or too weak, so that my search bar query still produces useful learning resources.

Acceptance Criteria

- External search is triggered automatically when internal results are insufficient.
- External results include title, link, source, and material type where possible.
- Internal and external results are visually distinguishable.
- Duplicate entries are removed or merged before display.

Technical Notes

- External fallback should happen only after internal sufficiency evaluation.
- The discovery agent should use allowlisted, education-relevant sources.
- Deduplication should compare normalized URLs, titles, and semantic similarity.
- Externally found resources must go through the same validation pipeline as internal items.

Story 2.3 — Material Type Detection

User Value Statement

As a learner, I want each result clearly labeled as video, article, book, or site, so that I can choose the format I prefer.

Acceptance Criteria

- Every material is assigned a type label.

- Type is visible in the results table.
- Users can filter by type.
- Ambiguous items can be flagged for later review.

Technical Notes

- Type detection can use a hybrid of rules and agent classification.
 - Confidence scores should be stored for ambiguous classifications.
 - The validation agent should verify consistency between type label and actual source.
-

Epic 3: Validation and Quality Assurance

Story 3.1 — Validate Relevance of Retrieved Materials

User Value Statement

As a learner, I want materials related to my search bar topic to be checked for relevance, so that I do not receive unrelated content.

Acceptance Criteria

- Every candidate material is evaluated by a validation agent.
- Materials receive a verdict such as approved, rejected, or uncertain.
- Rejected items are not shown by default.
- The final result set contains only validated or acceptable-confidence materials.

Technical Notes

- The validation agent should evaluate semantic relevance to the submitted topic and subtopics.

- Validation output should include score, verdict, and rationale.
 - Validation should consider more than titles, including extracted metadata and source context.
-

Story 3.2 — Validate Educational Quality

User Value Statement

As a learner, I want poor-quality resources filtered out, so that the results from my search feel curated and trustworthy.

Acceptance Criteria

- Materials are evaluated for educational usefulness.
- Quality checks include credibility, clarity, and coverage.
- Low-quality items are rejected or flagged.
- Quality indicators can be shown in the UI.

Technical Notes

- Quality assessment should combine heuristics with agent reasoning.
 - Signals may include readability, source authority, external reputation, and prior feedback.
 - The agent should produce interpretable quality sub-scores.
-

Story 3.3 — Retry Search When Validation Fails

User Value Statement

As a learner, I want the system to refine the search automatically when too many results fail validation, so that I still get a useful set of resources.

Acceptance Criteria

- If validation eliminates too many materials, a new search iteration is triggered.
- Retry logic adjusts search strategy rather than repeating the exact same query.
- Users are informed that the platform is refining results.
- The workflow ends gracefully if quality thresholds cannot be met.

Technical Notes

- The orchestrator should support bounded retry loops.
 - Retry strategies may include synonym expansion, topic decomposition, and broader source discovery.
 - Validation-failure reasons should be machine-readable and reused by the search agent.
-

Epic 4: Ranking and Learning Path Presentation

Story 4.1 — Rank Materials by Difficulty

User Value Statement

As a learner, I want materials ranked by difficulty, so that I can start at the right level after searching a topic.

Acceptance Criteria

- Each material receives a difficulty score or level.
- Users can sort by difficulty.
- Beginner users see easier materials first by default.
- Difficulty labels are consistent across content types.

Technical Notes

- Difficulty ranking should use academic level, readability, prerequisites, and external metadata.
 - The ranking agent should normalize inconsistent difficulty signals across sources.
 - Ranking should be stable and reproducible for the same material/query combination.
-

Story 4.2 — Display a Sortable Results Table

User Value Statement

As a learner, I want a table of results for the topic I searched, so that I can compare resources efficiently.

Acceptance Criteria

- Results are displayed in a table format.
- Columns include title, source, type, difficulty, quality/relevance score, and link.
- Users can sort and filter the table.
- Clicking a result opens the resource in a new tab.
- The table performs well with larger result sets.

Technical Notes

- Final assembled results should follow a normalized schema.
 - Sorting should use stored metadata, not rerun AI workflows.
 - Pagination or virtualization should be supported as data scales.
-

Story 4.3 — Explain Why a Material Was Recommended

User Value Statement

As a learner, I want a brief explanation for why each material was shown for my search, so that I can trust the recommendation.

Acceptance Criteria

- Each result includes a short explanation.
- The explanation references relevance, level, sequence, or quality.
- Explanations are concise and understandable.
- Additional detail can be expanded if needed.

Technical Notes

- Explanations should be grounded in structured retrieval, validation, and ranking signals.
 - The agent must avoid generic unsupported explanations.
 - Only known evidence from system outputs should be referenced.
-

Epic 5: Automated Problem Set Aggregator

Story 5.1 — Fetch Practice Problem Links from the Searched Topic

User Value Statement

As a learner, I want practice problems related to the topic I searched in the search bar, so that I can reinforce the concept through exercises.

Acceptance Criteria

- The system fetches relevant problems after a topic search.

- Supported sources include Codeforces, LeetCode, and AtCoder.
- Only problem links and metadata are displayed.
- Problems are shown in a dedicated section or tab.

Technical Notes

- The problem agent should map search topics to platform tags and canonical concepts.
 - The system must not scrape or display full problem statements.
 - Source metadata should be normalized across providers.
-

Story 5.2 — Rank Problems by Difficulty and Success Rate

User Value Statement

As a learner, I want practice problems ranked in a useful order, so that I can progress gradually from easier to harder exercises.

Acceptance Criteria

- Problems include difficulty and, where available, success rate.
- Users can sort problems by difficulty, source, or success rate.
- Difficulty is normalized across platforms.
- Missing metadata is handled gracefully.

Technical Notes

- A normalization layer should convert source-specific difficulty systems into one scale.
- Success-rate or acceptance-rate metadata should be used where available.
- If needed, the agent may infer approximate difficulty with confidence labeling.

Story 5.3 — Filter Problems by Platform

User Value Statement

As a learner, I want to filter practice problems by source platform, so that I can use sites I am comfortable with.

Acceptance Criteria

- Users can filter by Codeforces, LeetCode, AtCoder, and General Problems.
- Filtering updates the table dynamically.
- Every problem clearly shows its source.
- Problem links open directly in a new tab.

Technical Notes

- Provider-specific fetchers should feed a shared normalized schema.
- The UI should distinguish external linked problems from internally generated tasks.
- The architecture should allow future sources to be added easily.

Story 5.4 — General Problems Generated by the Agent

User Value Statement

As a learner, I want general problems when platform-specific ones are insufficient, so that I still have practice opportunities for the topic I searched.

Acceptance Criteria

- The platform can display general problems not tied to external sites.
- General Problems are clearly marked as agent-generated.
- Each includes title, topic focus, and difficulty.

- Generated problems are distinct from linked external problems.

Technical Notes

- Generated problems should be derived from the topic and learning-path context.
 - An evaluator agent should review generated problems before display.
 - These problems must be concise, implementation-oriented, and educationally aligned.
-

Epic 6: Foundational Task Generation

Story 6.1 — Generate Foundational Tasks from the Topic Search

User Value Statement

As a learner, I want foundational tasks such as “Implement BFS” after searching a topic, so that I can practice the core idea before harder exercises.

Acceptance Criteria

- The platform generates foundational tasks related to the searched topic.
- Tasks are non-story-based and implementation-focused.
- Each task includes a title and clear objective.
- Tasks are appropriate for the learner’s level.

Technical Notes

- The task-generation agent should synthesize tasks from validated materials and topic knowledge.
- Tasks should be canonical and reusable where possible.
- The system should avoid vague prompts and prioritize concrete algorithmic actions.

Story 6.2 — Order Foundational Tasks in a Learning Sequence

User Value Statement

As a learner, I want foundational tasks presented in a logical order, so that I can build understanding step by step.

Acceptance Criteria

- Tasks are ordered by prerequisite structure or difficulty.
- Simpler tasks appear before advanced variations.
- The recommended sequence is visible in the UI.
- Users can still skip or reorder tasks if desired.

Technical Notes

- Task order should be guided by concept dependencies and topic graph structure.
- Difficulty metadata and prerequisite tags should influence sequencing.
- The system should support future learning modes such as interview prep or academic study.

Epic 7: Code Editor Workspace

Story 7.1 — Dedicated Code Editor for Task Submission

User Value Statement

As a learner, I want a dedicated code editor space, so that I can write or paste solutions for review directly in the platform.

Acceptance Criteria

- Users can open a code editor from a task or problem.
- Users can paste or write code in the editor.
- Syntax highlighting is supported.
- The editor preserves temporary draft content.
- The architecture allows future compiler integration.

Technical Notes

- The editor, review panel, and metadata panel should be modular frontend components.
 - Submission payloads should include code, task context, language, and topic context.
 - The backend contract should be future-proofed for compile/run features.
-

Story 7.2 — Select Programming Language

User Value Statement

As a learner, I want to choose my programming language, so that the code review is accurate and language-aware.

Acceptance Criteria

- Users can select a language before submitting code.
- The selected language is visible in the editor workspace.
- Review output reflects the chosen language.
- Unsupported languages are handled clearly.

Technical Notes

- The review pipeline should use language metadata to select prompts and heuristics.
- Language selection should be part of the submission contract.

- The system should be designed for future language-specific compiler adapters.
-

Story 7.3 — Submit Code for AI Review

User Value Statement

As a learner, I want to submit my code and receive structured feedback, so that I can improve my solution.

Acceptance Criteria

- Users can submit code directly from the editor.
- The platform shows a review-in-progress state.
- Review results appear in categorized sections.
- Errors are communicated clearly if review cannot be completed.

Technical Notes

- The review workflow should be asynchronous at the app level.
 - Submission context should include the associated task/problem and searched topic.
 - Agent output should be structured JSON, not raw freeform text only.
-

Epic 8: AI Code Review Engine

Story 8.1 — Detect Bugs

User Value Statement

As a learner, I want the platform to identify likely bugs in my code, so that I can fix incorrect behavior quickly.

Acceptance Criteria

- Review output contains a Bugs category.
- Findings describe likely logical or implementation issues.
- Each finding references affected code areas.
- If no likely bugs are found, that is stated clearly.

Technical Notes

- Bug detection should combine agent reasoning with deterministic checks where possible.
 - Outputs should include code region references such as line ranges.
 - Findings should include confidence levels.
-

Story 8.2 — Identify Edge Cases

User Value Statement

As a learner, I want the system to point out edge cases I missed, so that my solution becomes more robust.

Acceptance Criteria

- Review output includes an Edge Cases category.
- Missing or risky corner cases are identified.
- Feedback includes concrete example scenarios.
- Suggested tests are relevant to the task.

Technical Notes

- Edge-case analysis should use both task context and code structure.
- The agent should reason about likely failing inputs and expected behavior.

- This workflow should support future execution-based validation.
-

Story 8.3 — Suggest Optimizations

User Value Statement

As a learner, I want optimization suggestions, so that I can improve performance after making the code correct.

Acceptance Criteria

- Review output includes an Optimizations category.
- Suggestions mention time or space improvements where relevant.
- Optional improvements are distinguished from critical issues.
- Feedback is task-aware, not generic.

Technical Notes

- Optimization analysis should compare the user's solution against expected algorithmic patterns.
 - Complexity expectations should be task-dependent.
 - Suggestions should be suppressed when no meaningful optimization exists.
-

Story 8.4 — Review Styling and Indentation

User Value Statement

As a learner, I want feedback on code style and indentation, so that my code is cleaner and easier to maintain.

Acceptance Criteria

- Review output includes a Styling/Indentation category.

- Style feedback is separated from correctness feedback.
- Concrete formatting issues are identified.
- Low-priority style comments do not dominate major issues.

Technical Notes

- Style review should be language-aware.
 - Findings should be normalized by severity.
 - Lightweight style analysis should avoid excessive token or compute cost.
-

Story 8.5 — Categorize All Review Suggestions

User Value Statement

As a learner, I want review suggestions grouped into clear categories, so that I can address issues systematically.

Acceptance Criteria

- Feedback is grouped into Bugs, Edge Cases, Optimizations, and Styling/Indentation.
- Categories can contain zero or more findings.
- Empty categories are displayed clearly when applicable.
- The UI makes categories visually distinct.

Technical Notes

- The code review agent must return stable structured categories.
 - Backend post-processing should normalize inconsistent outputs into a standard schema.
 - Category ordering should be fixed for UI consistency.
-

Epic 9: Review UX and Code Mapping

Story 9.1 — Highlight Code on Suggestion Hover

User Value Statement

As a learner, I want relevant code to highlight when I hover over a suggestion, so that I can immediately see what the feedback refers to.

Acceptance Criteria

- Hovering over a suggestion highlights the related code block.
- Highlighted regions are visually distinct.
- Different suggestions can map to different code regions.
- The interaction remains smooth during editing.

Technical Notes

- Review outputs must include line ranges, character spans, or AST-based anchors.
 - The frontend should support mapping from suggestion panel to code editor regions.
 - Approximate mapping should be labeled as such when exact anchoring is uncertain.
-

Story 9.2 — Show Related Suggestions When Code Is Selected

User Value Statement

As a learner, I want to click code and see which review suggestions apply to it, so that I can inspect feedback contextually.

Acceptance Criteria

- Clicking or selecting code shows related review comments.
- If no review comments apply, the UI states that clearly.

- Multiple comments for the same region can be grouped.
- Interaction remains usable after moderate code edits.

Technical Notes

- The mapping system should support reverse lookup from code region to findings.
 - Review data should be version-aware to avoid misleading mappings after edits.
 - This structure prepares the product for inline review comments later.
-

Story 9.3 — Mark Feedback as Stale After Edits

User Value Statement

As a learner, I want to know when feedback may be outdated after editing my code, so that I do not rely on stale suggestions.

Acceptance Criteria

- Editing code after review marks the feedback as potentially outdated.
- Users are prompted to resubmit for an updated review.
- Old feedback remains accessible until replaced.
- The stale-state message is clear but unobtrusive.

Technical Notes

- Each review should be tied to a specific code snapshot or hash.
 - Editing invalidates exact mapping confidence.
 - The system should preserve prior review history while marking it stale.
-

Epic 10: Agent Orchestration and Workflow Reliability

Story 10.1 — Orchestrate Specialized AI Agents

User Value Statement

As a platform owner, I want specialized agents to handle search, validation, ranking, generation, and review, so that the system remains modular and maintainable.

Acceptance Criteria

- The workflow is separated into clear agent stages.
- Each stage produces outputs consumable by the next stage.
- Partial failures are handled gracefully.
- Execution traces exist for debugging.

Technical Notes

- Suggested agents: search agent, validation agent, ranking agent, task generator, problem aggregator, code reviewer, evaluator/guard agent.
 - Contracts between stages should be strongly typed.
 - Agents should be replaceable without rewriting the entire workflow.
-

Story 10.2 — Retry and Recovery Logic

User Value Statement

As a platform owner, I want agent failures to recover intelligently, so that users do not experience brittle workflows.

Acceptance Criteria

- Search, validation, and ranking steps can retry when appropriate.

- Retry behavior is bounded and logged.
- Users see meaningful fallback messaging.
- Internal errors are not exposed directly to end users.

Technical Notes

- Failure types should be classified as transient, low-confidence, or terminal.
 - Intermediate artifacts should be preserved to avoid unnecessary recomputation.
 - Retry policies should vary by stage and failure type.
-

Story 10.3 — Structured Output Contracts for All Agents

User Value Statement

As an engineering team, we want every agent to return structured output, so that frontend rendering and downstream processing remain predictable.

Acceptance Criteria

- Each agent returns data in a predefined schema.
- Invalid or partial payloads are caught before rendering.
- The UI does not depend on raw model prose.
- Schema changes can be versioned safely.

Technical Notes

- Strongly typed response models should be used across the orchestrator.
 - Confidence, rationale, and failure reasons should be part of schemas where relevant.
 - Validation layers should sit between model output and persistence/UI.
-

Epic 11: Trust, Explainability, and Guardrails

Story 11.1 — Show Source Provenance and Confidence

User Value Statement

As a learner, I want to understand where results come from and how reliable they are, so that I can trust the platform appropriately.

Acceptance Criteria

- Materials and problems display their source.
- Confidence or validation indicators can be shown where appropriate.
- Generated tasks are clearly labeled as generated.
- Users can inspect brief recommendation rationale.

Technical Notes

- Provenance should be tracked end-to-end through the agent workflow.
- Confidence indicators should be derived from structured system outputs.
- The architecture should support future audit views for admins.

Story 11.2 — Guard Against Low-Quality or Fabricated Results

User Value Statement

As a platform owner, I want safeguards on AI-generated and AI-selected output, so that users receive dependable educational content.

Acceptance Criteria

- Low-quality, broken, or irrelevant results are filtered or flagged.
- The system does not invent external sources or links.
- Code review does not overstate uncertain claims as facts.
- Unsafe or malformed outputs are blocked from display.

Technical Notes

- A guard/evaluator agent should run after generative steps.
 - Link display must be restricted to verified retrieved URLs.
 - Uncertain code review findings should be labeled probabilistically.
-

Epic 12: Admin Controls and Continuous Improvement

Story 12.1 — Admin Review and Correction of Material Metadata

User Value Statement

As an admin or professor, I want to review and correct resource metadata, so that future search bar results improve over time.

Acceptance Criteria

- Admins can inspect stored resources and metadata.
- Admins can edit tags, difficulty, and type.
- Corrections affect future ranking and retrieval.
- Low-confidence items are easy to identify.

Technical Notes

- Human corrections should be persisted as overrides or authoritative labels.
 - Admin edits should become signals for future ranking/validation improvements.
 - Low-confidence classifications should be surfaced for manual review queues.
-

Story 12.2 — Capture User Feedback on Recommendations

User Value Statement

As a learner, I want to mark results as helpful or unhelpful, so that the platform improves its future recommendations.

Acceptance Criteria

- Users can provide positive or negative feedback on materials and problems.
- Feedback can optionally include a reason.
- The UI confirms feedback submission.
- Feedback influences future ranking and quality evaluation.

Technical Notes

- Feedback should be linked to topic, resource, and user context.
 - The ranking agent should be able to consume aggregated feedback signals.
 - Negative feedback categories can be used to improve retry and validation logic.
-

Epic 13: Platform Foundations and Scalability

Story 13.1 — Observe and Trace Agent Workflows

User Value Statement

As an engineering team, we want observability into agent workflows, so that we can debug poor recommendations and optimize performance.

Acceptance Criteria

- Each orchestration run is traceable.
- Stage-level execution times and errors are logged.
- Logs can be tied to user actions such as specific search bar submissions.
- Sensitive information is handled appropriately in observability systems.

Technical Notes

- Stage-level telemetry should be stored in structured form.
 - Inputs and outputs should be logged with privacy-aware redaction.
 - Observability is essential for evaluating multi-agent production behavior.
-

Story 13.2 — Maintain a Scalable Data Model

User Value Statement

As a platform owner, I want a scalable backend data model, so that materials, problems, tasks, and reviews can grow with the product.

Acceptance Criteria

- Materials, problems, tasks, code submissions, and reviews are persisted in normalized models.
- Relationships between topic, resource, submission, and review are preserved.
- The data model supports analytics and future search improvements.

- Internal curated content and externally discovered content are distinguishable.

Technical Notes

- Recommended entities include resources, topics, tags, orchestration runs, submissions, and review artifacts.
 - Validation scores, ranking signals, and embeddings should be stored independently from raw source records.
 - Provider-specific fields should be isolated behind extensible schemas.
-

Suggested MVP Scope

A strong MVP would include:

- authentication and role setup
 - **search bar** for topic queries
 - internal search + web fallback
 - validation agent
 - difficulty ranking
 - sortable materials table
 - problem links aggregator
 - foundational task generation
 - code editor
 - categorized AI code review
-

Recommended Delivery Order

1. Authentication and profile
2. Search bar and topic orchestration
3. Internal retrieval and web fallback
4. Validation and ranking
5. Results table
6. Problem aggregation
7. Foundational tasks
8. Code editor and review
9. Review interactions
10. Admin, trust, and scale features